

Reviewer Pack — Discrete Morse Betti Growth in Resolution-Frege Triangle Com...

Entry 3b8307405b8f · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-05-19 19:08:22 UTC

Discrete Morse Betti Growth in Resolution-Frege Triangle Complexes for PHP

Entry ID: 3b8307405b8f **Verdict:** INCONCLUSIVE **Recorded:** 2026-05-19 19:08:22 UTC

1. Conjecture

Field A (mathematical branch): Discrete Morse theory and cellular homology of 2-CW complexes (Forman 1998; Benedetti-Lutz) **Field B** (complexity object): Resolution refutations of the Pigeon-hole Principle $\text{PHP}_{\{n+1 \rightarrow n\}}$ as a sub-system of Frege/Extended Frege

Statement:

Given any resolution refutation P of $\text{PHP}_{\{n+1 \rightarrow n\}}$, define its triangle 2-CW complex $K(P)$: 0-cells are the distinct clauses occurring in P , 1-cells are unordered pairs $\{C, D\}$ of clauses that co-appear as inputs/output of at least one resolution step, and 2-cells are triangular faces $\{A, B, R\}$ for each resolution inference $A, B \vdash R$. Let $\beta_1(K(P); F_2) = (|E| - |V| + \beta_0) - \text{rank}_{\{F_2\}}(\partial_2)$ be the first mod-2 Betti number. Conjecture: for every $n \geq 3$ and every (tree-like or DAG) resolution refutation P of $\text{PHP}_{\{n+1 \rightarrow n\}}$, $\beta_1(K(P); F_2) \geq \lfloor n \cdot \log_2(n+1) \rfloor$, and moreover the ratio $\beta_1(K(P))/|P|$ stabilizes to a strictly positive constant $c_{\text{PHP}} > 0$ as n grows; a single instance with $\beta_1 < \lfloor n \cdot \log_2(n+1) \rfloor$ refutes the conjecture.

Rationale (proposer's reasoning):

Resolution proofs are routinely measured by size/width but almost never by the cellular topology of the inference DAG itself; treating each resolution rule as a glued triangle yields a 2-complex whose homology is a functorial invariant of the proof object, not of the encoded truth-table — sidestepping the Razborov-Rudich natural-proofs trap that killed prior single-number truth-table attempts in this approach. The construction is purely combinatorial (no arithmetization, no oracle), so it does not algebrize or relativize. If β_1 genuinely scales with n it provides a Frege-relevant complexity lower bound complementing Haken's width method while exposing a previously unmeasured topological resource.

Taxonomy category: PROOF_COMPLEXITY_EXT_FREGE (status at proposal time: alive)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): 079cf3640d98ff43

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

Across 90 runs ($n \in \{3, 4, 5\}$, 30 seeds each), conjecture is SUPPORTED iff every run satisfies $\beta_1(K(P); F_2) \geq \lfloor n \cdot \log_2(n+1) \rfloor$ AND $\text{mean}(\beta_1/|P|)$ at $n=5 \geq \text{mean}$ at $n=3$ with all three per- n means > 0.01 and $\text{variance}(\beta_1/|P|)$ at $n=5 \leq \text{variance}$ at $n=3$.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.92	SAFE	SAFE
NATURAL_PROOFS	SAFE	0.95	SAFE	SAFE
ALGEBRIZATION	SAFE	0.95	SAFE	SAFE
KARP_LIPTON	SAFE	0.95	SAFE	SAFE

4. Novelty audit

Verdict: NOVEL against 9 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - resolution refutation pigeonhole principle topological complexity Betti - discrete Morse theory proof complexity CNF resolution DAG - simplicial complex resolution proof PHP lower bound homology

Top relevant hits considered: - [<http://arxiv.org/abs/1608.07792v1>] A Generalized Resolution Proof Schema and the Pigeonhole Principle - [<http://arxiv.org/abs/cs/0304041v1>] $P \neq NP$, propositional proof complexity, and resolution lower bounds for the weak pigeonhole principle - [<http://arxiv.org/abs/0909.5038v1>] An Exponential Lower Bound on OBDD Refutations for Pigeonhole Formulas - [<http://arxiv.org/abs/2504.15729v2>] Strong discrete Morse theory - [<http://arxiv.org/abs/2603.28382v1>] Anick Resolution for Lawvere Theories from Algebraic Discrete Morse Theory - [<http://arxiv.org/abs/1804.02478v5>] Complexity of the CNF-satisfiability problem - [<http://arxiv.org/abs/1801.08709v2>] Adaptive Lower Bound for Testing Monotonicity on the Line - [<http://arxiv.org/abs/2110.02458v3>] Magnitude homology of graphs and discrete Morse theory on Asao-Izumihara complexes

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, $\leq 90s$ wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def gaussian_elimination(matrix):
        rows, cols = len(matrix), len(matrix[0])
        for i in range(rows):
            max_row = i + max(range(i, rows), key=lambda r: abs(matrix[r][i]))
            matrix[i], matrix[max_row] = matrix[max_row], matrix[i]
            for j in range(cols):
                if j != i:
                    factor = matrix[j][i] / matrix[i][i]
                    for k in range(rows):
```

```

        matrix[j][k] -= factor * matrix[i][k]
rank = sum(1 for row in matrix if any(row))
return rank

def compute_beta_1(vertices, edges, triangles):
    V, E, F = len(vertices), len(edges), len(triangles)
    beta_0 = 1
    boundary_matrix = [[0] * (E + F) for _ in range(E)]
    for i, (C, D) in enumerate(edges):
        boundary_matrix[i][i] = -1
        boundary_matrix[i][i + E] = 1
    for j, (A, B) in enumerate(triangles):
        boundary_matrix[j + E][j] = -1
        boundary_matrix[j + E][j + F] = 1
    rank_boundary = gaussian_elimination(boundary_matrix)
    return (E - V + beta_0) - rank_boundary

def dpll(clauses, assignment):
    if not clauses:
        return True
    clause = next(c for c in clauses if any(var in c for var in assignment))
    pos_var = next(var for var in clause if var not in assignment)
    neg_var = -pos_var

    # Try assigning the positive literal
    new_assignment = assignment.copy()
    new_assignment[pos_var] = True
    if dpll([c for c in clauses if not (set(c) & {pos_var, neg_var})], new_assignment):
        return True

    # Try assigning the negative literal
    new_assignment[neg_var] = True
    if dpll([c for c in clauses if not (set(c) & {neg_var, pos_var})], new_assignment):
        return True

    return False

def parse_php(n):
    clauses = []
    for i in range(1, n + 2):
        clause = [i]
        for j in range(i + 1, n + 2):
            clause.append(-j)
        clauses.append(clause)
    return clauses

n_values = [3, 4, 5]
results = []

for n in n_values:
    clauses = parse_php(n)
    instances_tested = 0
    beta_1_sum = 0

    for _ in range(30):
        variable_ordering = random.sample(range(1, n + 2), n)

```

```

assignment = {var: False for var in variable_ordering}

if dpll(clauses, assignment):
    instances_tested += 1
    beta_1 = compute_beta_1(variable_ordering, [], [])
    beta_1_sum += beta_1

if beta_1 < n * math.log2(n + 1):
    return {
        "metric_name": "beta_1",
        "metric_value": beta_1,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": f"n={n}, beta_1={beta_1} < {n * math.log2(n + 1)}"
    }

if instances_tested == 0:
    return {
        "metric_name": "beta_1",
        "metric_value": None,
        "instances_tested": instances_tested,
        "conjecture_holds": False,
        "counterexample": "No valid resolution found"
    }

beta_1_avg = beta_1_sum / instances_tested
results.append((n, beta_1_avg))

if not all(beta_1 >= n * math.log2(n + 1) for n, beta_1 in results):
    return {
        "metric_name": "beta_1",
        "metric_value": None,
        "instances_tested": sum(1 for _, _ in results),
        "conjecture_holds": False,
        "counterexample": "Some beta_1 values are less than n * log2(n + 1)"
    }

if len(results) < 3:
    return {
        "metric_name": "beta_1",
        "metric_value": None,
        "instances_tested": sum(1 for _, _ in results),
        "conjecture_holds": False,
        "counterexample": "Not enough data points"
    }

beta_1_means = [beta_1 for n, beta_1 in results]
beta_1_variances = [(beta_1 - sum(beta_1_means) / len(beta_1_means)) ** 2 for beta_1 in beta_1_means]

return {
    "metric_name": "beta_1",
    "metric_value": sum(beta_1_means) / len(beta_1_means),
    "instances_tested": sum(1 for _, _ in results),
    "conjecture_holds": True,
    "counterexample": ""
}

```

```

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] if sys.argv[1:] else list(range(2, 89, 3))

    results = []
    for seed in seeds:
        result = run_trial(seed)
        print(f"TRIAL: {result}")
        results.append(result)

    beta_1_values = [r["metric_value"] for r in results if r["metric_value"] is not None]
    instances_tested = sum(r["instances_tested"] for r in results)
    conjecture_holds = all(r["conjecture_holds"] for r in results)
    support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)

    if conjecture_holds and support_fraction >= 0.8:
        print(f"RESULT: SUPPORTED mean={sum(beta_1_values) / instances_tested} std={math.sqrt(sum(
    elif not conjecture_holds:
        first_failing_seed = next(r["seed"] for r in results if not r["conjecture_holds"])
        print(f"RESULT: FALSIFIED counterexample=\"beta_1 < n * log2(n + 1)\" first_failing_seed={
    else:
        print(f"RESULT: INCONCLUSIVE support_fraction={support_fraction}")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

```

--STDERR--
Traceback (most recent call last):
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ae02aa7.py", line 149, in <module>
    result = run_trial(seed)
              ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ae02aa7.py", line 88, in run_trial
    if dpll(clauses, assignment):
    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ae02aa7.py", line 57, in dpll
    if dpll([c for c in clauses if not (set(c) & {pos_var, neg_var})], new_assignment):
    ~~~~~^
  File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_7ae02aa7.py", line 51, in dpll
    pos_var = next(var for var in clause if var not in assignment)
    ~~~~~^
StopIteration

```

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test crashed with a StopIteration exception in the DPLL routine before producing any data; no RESULT line or measurements were emitted, so neither the support nor falsification conditions can be evaluated. | next: Fix the DPLL unit-propagation bug (guard the next(...) call against fully-assigned clauses) and re-run the 90-trial pre-registered protocol to obtain β_1 statistics.

11. Audit log (LLM calls)

Total LLM calls: 9

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	claude_max	opus	0	0	159837
2	preregistration	claude_max	opus	0	0	7768
3	novelty	claude_max	opus	0	0	3937
4	novelty	claude_max	opus	0	0	8271
5	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	24034
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16114
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17990
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	17858
9	judge	claude_max	opus	0	0	4412

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 260222 ms total latency. Provider mix: {'claude_max': 5, 'ollama_remote': 4}

(full prompt+response transcripts available in `research/audit/3b8307405b8f.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/3b8307405b8f.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/3b8307405b8f.tar.gz` (if generated)